Reinforcement Learning for a Ball Balancing Game

Ognjen Baucal
Computer science, Faculty of Science
University of Novi Sad

1 Problem description

1.1 Gameplay

The game consists of a ramp and a ball in a two-dimensional space. The objective of the game is to balance the ball on the ramp and keep it as close to the center of the ramp as possible. The game lasts ten seconds if the ball doesn't fall off the ramp before those ten seconds pass. The player has control of the ramps tilt, which can be adjusted in every frame, sixty frames per second. At the start of the game, the ball is randomly placed on the ramp with no initial velocity and the ramp's incline angle is set to zero degrees.

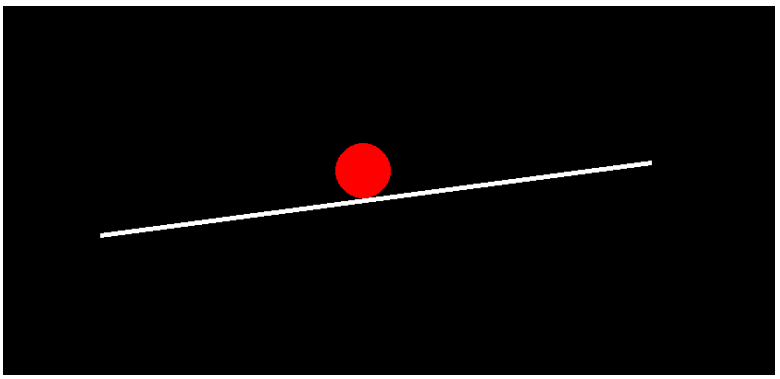


Figure 1: A snapshot of the in-game screen

1.2 State space

The first input is the incline angle of the ramp, the angle varies from [-85, 85] degrees. The second input is the distance between the ball and the center of the ramp. The distance is negative if the ball is on the left side of the ramp, positive if the ball is on the right side of the ramp. This way the agent gets the position and the distance to the center of the ramp, where the ball needs to be, in just one parameter. It's a more elegant and simple way than passing the x and y coordinates of the ball. The distance varies from [-300, 300]. The third and the fourth inputs are the horizontal and vertical velocity of the ball, the horizontal velocity ranges from [-900, 900], the vertical velocity ranges from [-1100, 1100].

The inputs vary in sizes which can have a significant impact on the performance and the training process of the neural network. It can lead to a bias during learning, vanishing or exploding gradients, slow convergence and it can make the choice of the hyperparameters and initial weights and biases of the neural network much more sensitive, which complicates the tuning process. Therefore the inputs are all scaled down to a range of [-1, 1].

1.3 Action space

The action space is discrete and it consists of three options: do nothing, increase the incline angle of the ramp and decrease the incline angle of the ramp. The ramp's angle can at most be changed by 90 degrees per one second, which is 1.5 degrees per frame.

1.4 Rewards

The agent receives two types of rewards, the time reward and the distance reward. The time reward is granted every frame, encouraging the agent to keep the ball on the ramp for the whole duration of the episode. The distance reward is given to the agent every frame, but its value increases the closer the ball is to the center of the ramp, encouraging the agent to keep the ball closer to the center. The agent also receives a punishment when the ball falls off the ramp.

2 Solution

2.1 Proximal Policy Optimization

Proximal Policy Optimization (PPO) is a reinforcement learning algorithm that optimizes a policy. It improves the stability of training using a clipped objective function, which prevents large updates to the policy. It works by collecting data from an episode with the current policy, estimating advantages and updating the policy with a loss function. The advantages are estimated by taking the difference of the discounted sum of rewards and the expected return value from the state predicted by a separate value neural network. For a given state s and action a, the clipped loss function is:

$$r = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta old}(a_t|s_t)}$$

$$L_{policy} = E_t\left[min(r \cdot A_t,\ clip(r,\ 1 - \epsilon,\ 1 + \epsilon) \cdot A_t)\right]$$

$\pi_\theta(a_t|s_t)$ - the probability of the action a_t in a state s_t given by the policy neural network with parameters θ

A_t - the estimated advantage

ϵ - a small hyperparameter that prevents the ratio from being too large or too small

For a value neural network the loss function is:

$$L_{value} = E_t[\frac{1}{2}(V(s_t) - R_t)^2]$$

$V(s_t)$ - the predicted value for the state s_t from the value neural network

R_t - the discounted sum of rewards

The discounted sum of rewards is calculated by the following formula:

$$R_t = \sum_{k=0}^{T-t} \gamma^k \cdot r_{t+k}$$

where r_{t+k} is the reward at time step $t+k$ and γ is the discount factor.

2.2 Neural network architecture

The policy neural network is a fully connected neural network with an input layer of four neurons, two hidden layers, and an output layer with 3 neurons. The output layer uses the Softmax activation function which returns the probability of each action, the other layers use the ReLU activation function.

The value neural network has the same structure as the policy neural network, except the output layer has one neuron. The output layer has no activation function, while the other layers use the ReLU activation function.

2.3 Training

The agent plays out the episode and records the states, actions, rewards and probabilities of the taken actions until it reaches a terminal state. Then the return at each time step is calculated and is then used to calculate the advantage at each time step. Now divide the collected experiences from the episode into batches of experiences, the batch size used was 30 since the better the player got the longer the episode lasts which is at most 600 frames. For each batch, a policy loss function and a value loss function is calculated and applied to the neural networks using the Adam optimizer. The hyperparameters ϵ and γ were set to 0.2 and 0.99 respectively.

3 Experiment

3.1 Experiment description

I trained four models each on 6000 episodes, which was an arbitrarily chosen number of episodes, where each model had either a different size or a different learning rate.

The first and the second model had 256 neurons in each hidden layer, while the third and the fourth model had 512 neurons in each hidden layer. The first and third model had a learning rate of 1e-3 while the second and fourth had a learning rate of 1e-5.

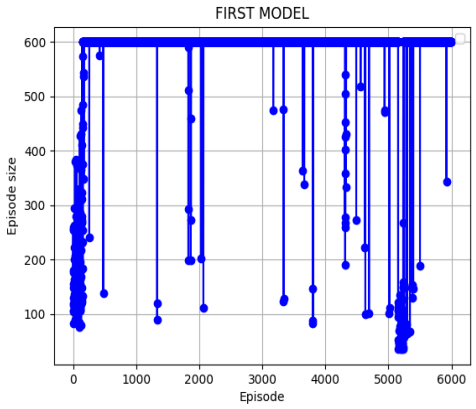
3.2 Experiment results

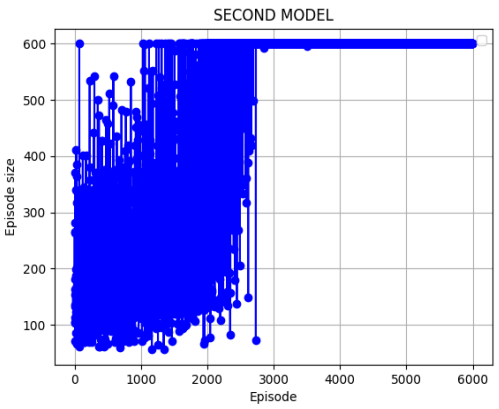
At the end of the training, all four models became very good at playing the game with some minor differences. Every model except the fourth balances the ball a little bit off center. I don't know for sure what caused this to happen, my guess is that my self made environment probably has physics engine issues causing the agent to sometimes get invalid states. On the graphs below, that graph episode sizes, we can see sudden changes in episode lengths which is probably what caused the irregularity in the playstyle of the models, because as we can see the fourth model didn't have any irregularities on its graph.

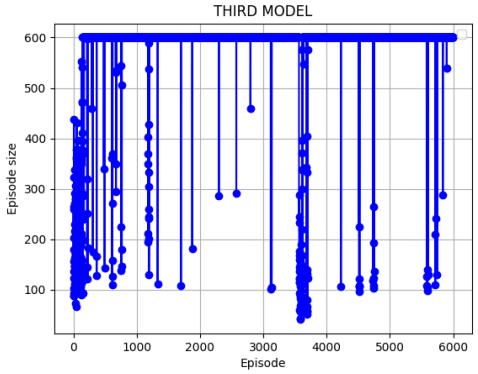
What's more important is that we can clearly see how the models with a smaller learning rate did much better at converging the policy loss and the value loss, while also being way more consistent at the game. Then if we compare the second and the fourth model we can see how the model with more neurons understood the complexity of the game much faster and adapted its playstyle faster.

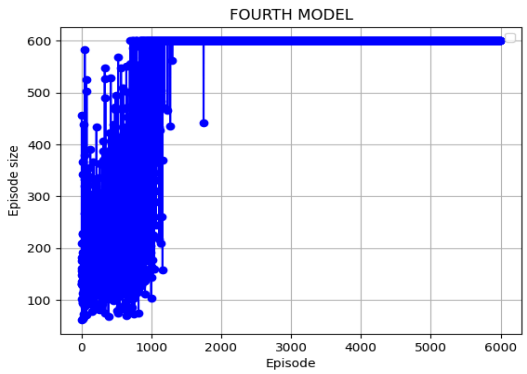
Except for the expected result, which was the fourth model being the best, there was a significant difference in the gameplay between the models with different learning rates before they got to 6000 episodes. When the ball would be placed at the end of the ramp, the model with a bigger learning rate would aggressively tilt the ramp trying to get the ball to the center as fast as possible, which would end up in the ball getting too much speed and flying over the center, usually falling off the ramp as the result. Meanwhile when the model with a smaller learning rate would be placed in that same situation, it would slowly get the ball to the center, taking more time but also taking no risk in overshooting the center. So in the given situation the model with a smaller learning rate is better, but if the situation changes and the ball gets placed close to the center of the ramp, the model with a bigger learning rate would get the ball to the center faster and therefore earn a bigger reward then the model with a smaller learning rate. which would again take its time to get the ball to the center, making the model with the bigger learning rate better at this situation.

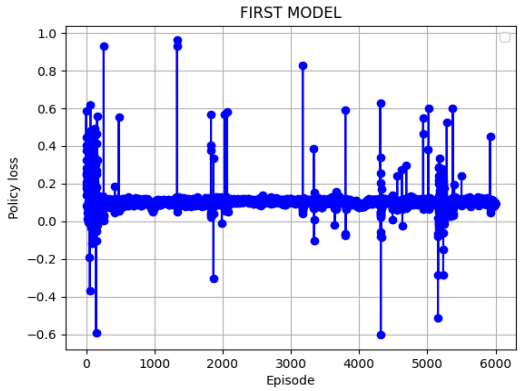
3.3 Visualisation

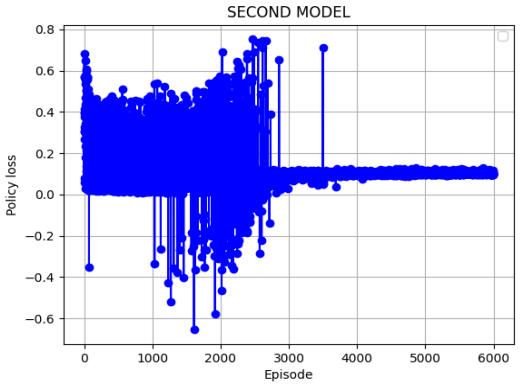


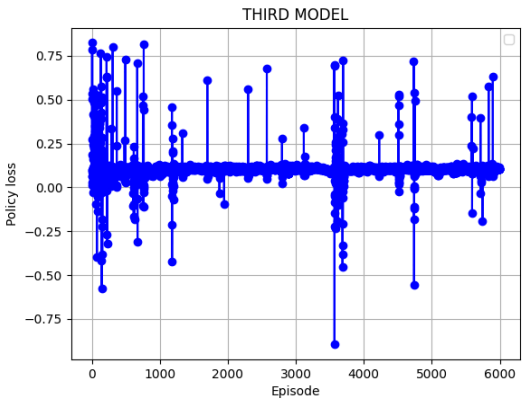


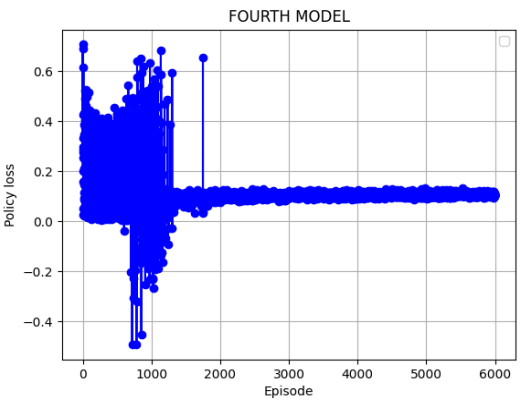


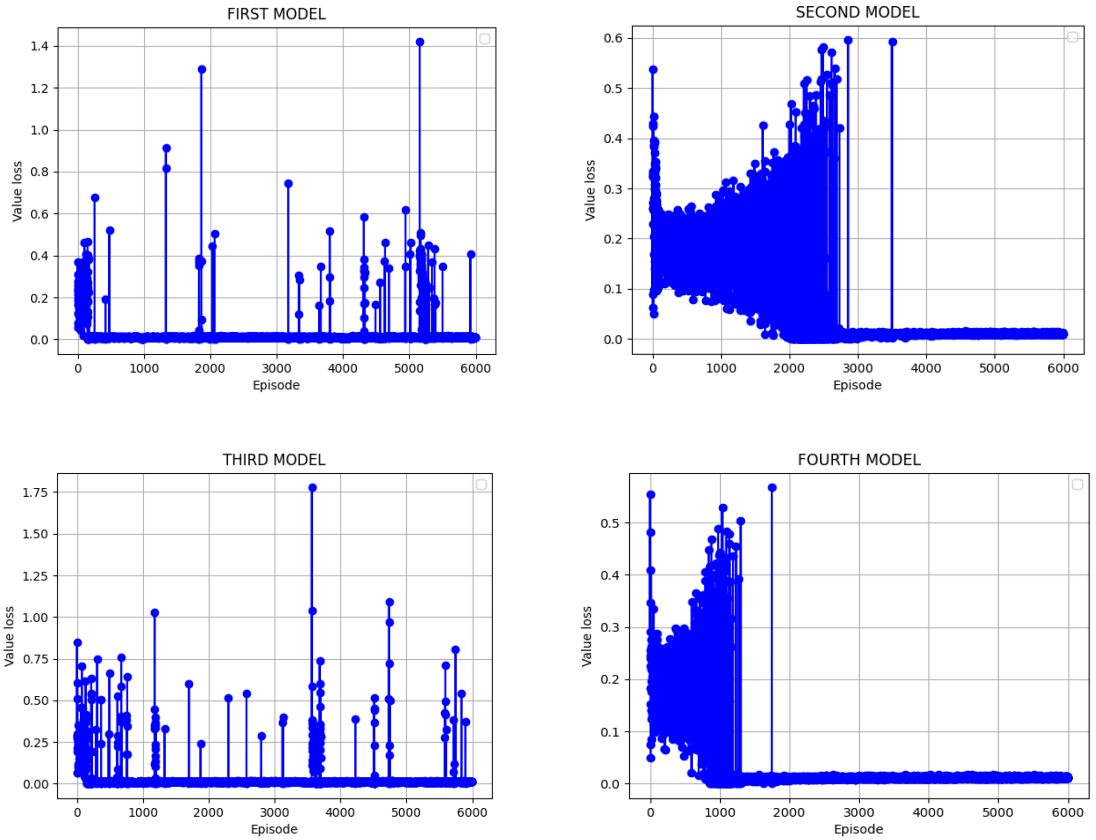












4 Conclusion

As stated before, the fourth model with the smaller learning rate and a bigger neural network was the best model, converging after about 1300 episodes and resulting in a perfect player at the game. If you are looking for a decent player instead of a perfect player, because of time or computational restrictions when it comes to training, the model with a bigger learning rate would be your best choice. Because of the different playstyles of the models with different learning rates, it seems like a bigger learning rate makes a more aggressive player that goes for a high risk high reward playstyle, while a smaller learning rate makes a cautious player that doesn't take risks and plays safely.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. "Proximal Policy Optimization Algorithms". In: arXiv:1707.06347v2 [cs.LG] 28 Aug 2017.